CLOUD COMPUTING MINI PROJECT

Study Planner & To-Do App

A Streamlit-based web application for managing study tasks, deadlines, and priorities

Submitted by	Apurwa
Roll Number	06
Project Guide	Shraddha Mukkawar
College	ISBM College of Engineering, Nande, Pune
Department	Artificial Intelligence and Data Science (AI & DS)
Academic Year	2026–2027

Submitted in partial fulfilment of the requirements for the Cloud Computing mini project.

Abstract

The Study Planner & To-Do App is a lightweight web application designed to help students organise academic tasks, assign due dates and priorities, and monitor their progress. It is built with Python and Streamlit and deployed as a web app through Streamlit Community Cloud. The application demonstrates cloud-hosted application delivery: users can access the interface through a browser without installing the application locally. The current version stores task data in Streamlit session state, so it is intended as a demonstration and does not guarantee permanent storage across session resets or restarts.

1. Introduction

Students often manage assignments, revision sessions, and project deadlines across multiple subjects. A simple task planner provides one place to record work and track progress. This project applies cloud computing concepts by developing a browser-accessible application and deploying it on a managed cloud platform.

2. Objectives

• Create a simple and user-friendly study task management interface.
• Allow users to record task names, subjects, due dates, priorities, and optional notes.
• Display task progress through summary metrics.
• Support filtering, sorting, completion, and deletion of tasks.
• Deploy the app to Streamlit Community Cloud and understand the role of a managed application platform.

3. Technologies Used

Python: programming language used for the app logic.
Streamlit: Python framework for building the interactive web interface.
GitHub: source-code repository used to host the project files.
Streamlit Community Cloud: hosting platform used to publish the app from the repository.
Browser: client used to access the deployed application.

4. System Features

• Add a task with a title, subject, due date, priority, and notes.
• View task counts for total, completed, pending, and overdue tasks.
• Filter tasks by status and sort them using available options.
• Mark tasks as completed or delete individual tasks.
• Clear completed tasks when they are no longer needed.

5. System Workflow

1. The project code and dependency file are stored in a GitHub repository.
2. Streamlit Community Cloud connects to the repository and runs the app entry-point file (`app.py`).
3. The platform provides a hosted web address for browser access.
4. A user enters task details and submits the form.
5. The app updates its task list and summary metrics in the active session.

6. Screenshots and Results

The following screenshots document the deployed application, task entry, dashboard metrics, task completion, and deployment configuration.

Screenshot 1

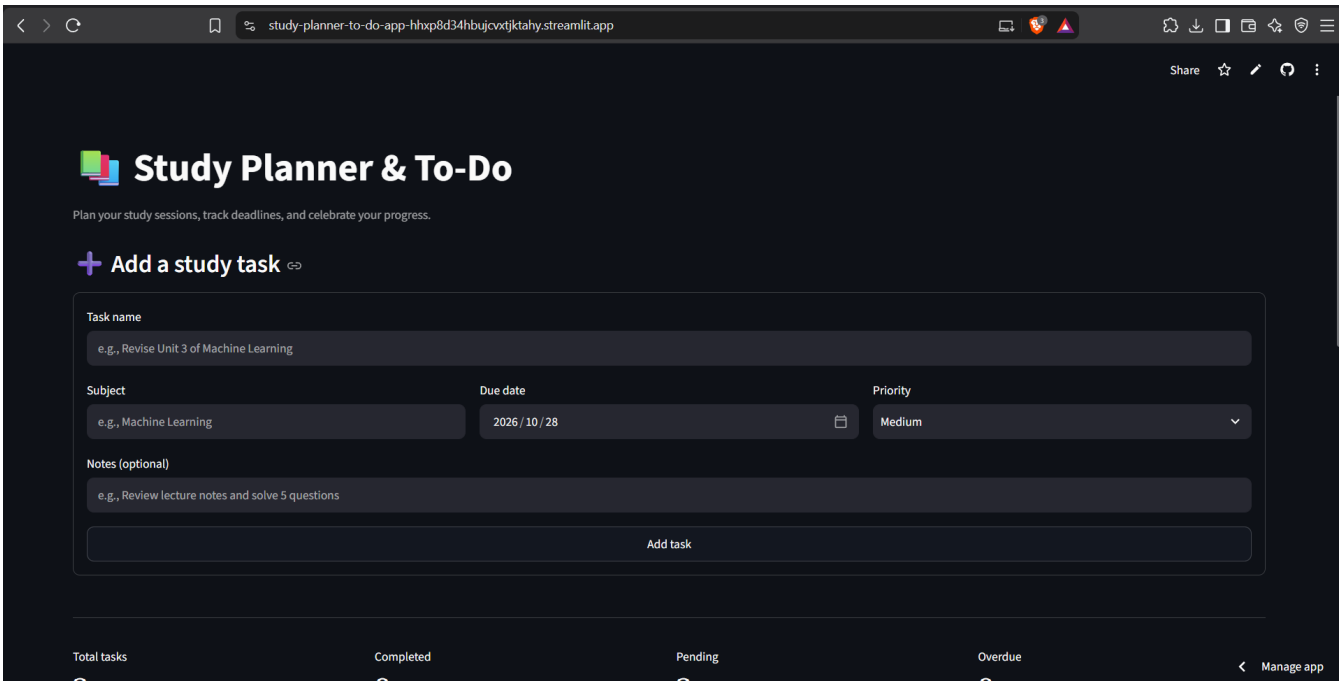


Figure 1. Deployed Study Planner & To-Do application homepage.

Screenshot 2

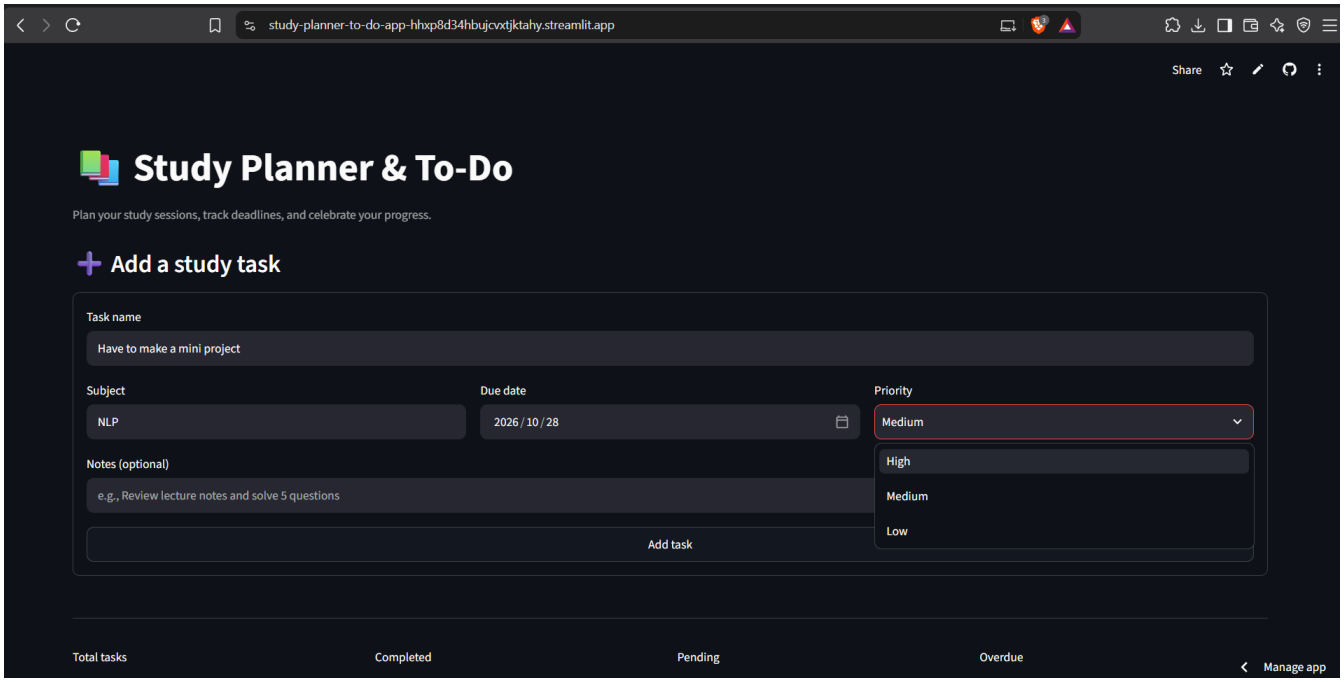


Figure 2. Task entry form with sample task details and priority selection.

Screenshot 3

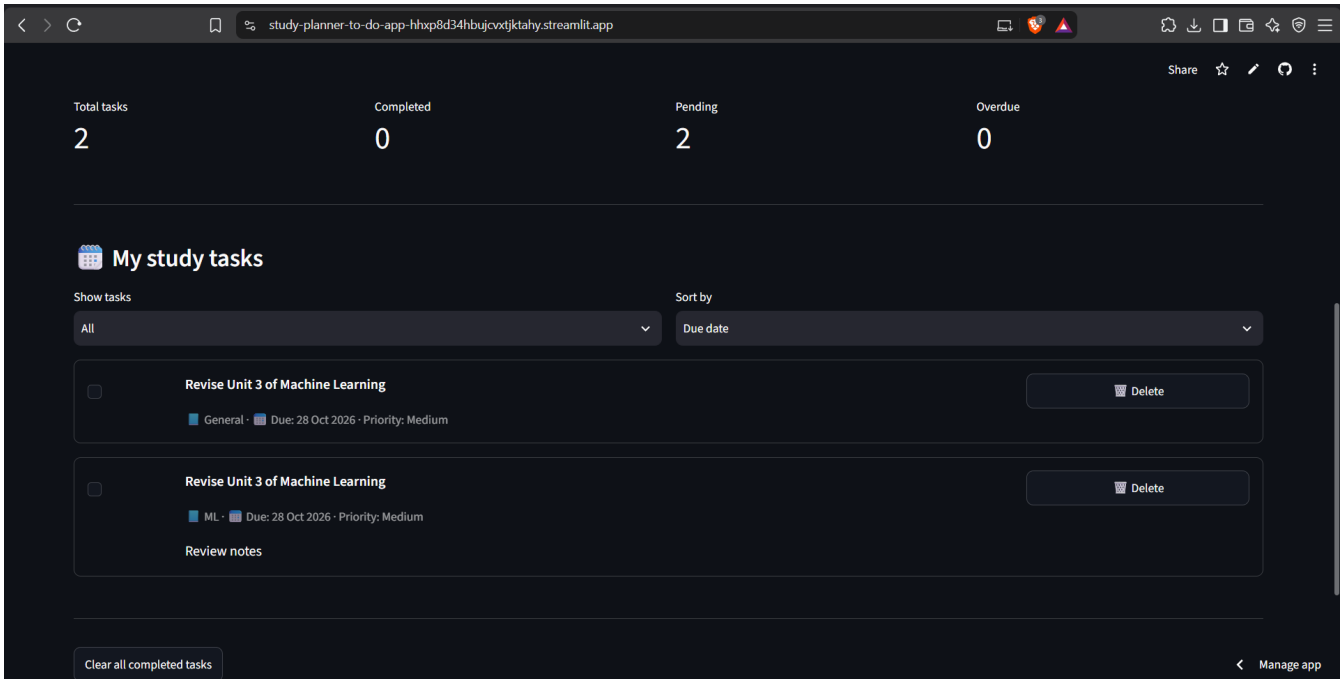


Figure 3. Task dashboard showing total, completed, pending, and overdue counts.

Observed dashboard state in this screenshot: 2 total tasks, 0 completed, 2 pending, and 0 overdue. The counts change as the user updates task status.

Screenshot 4

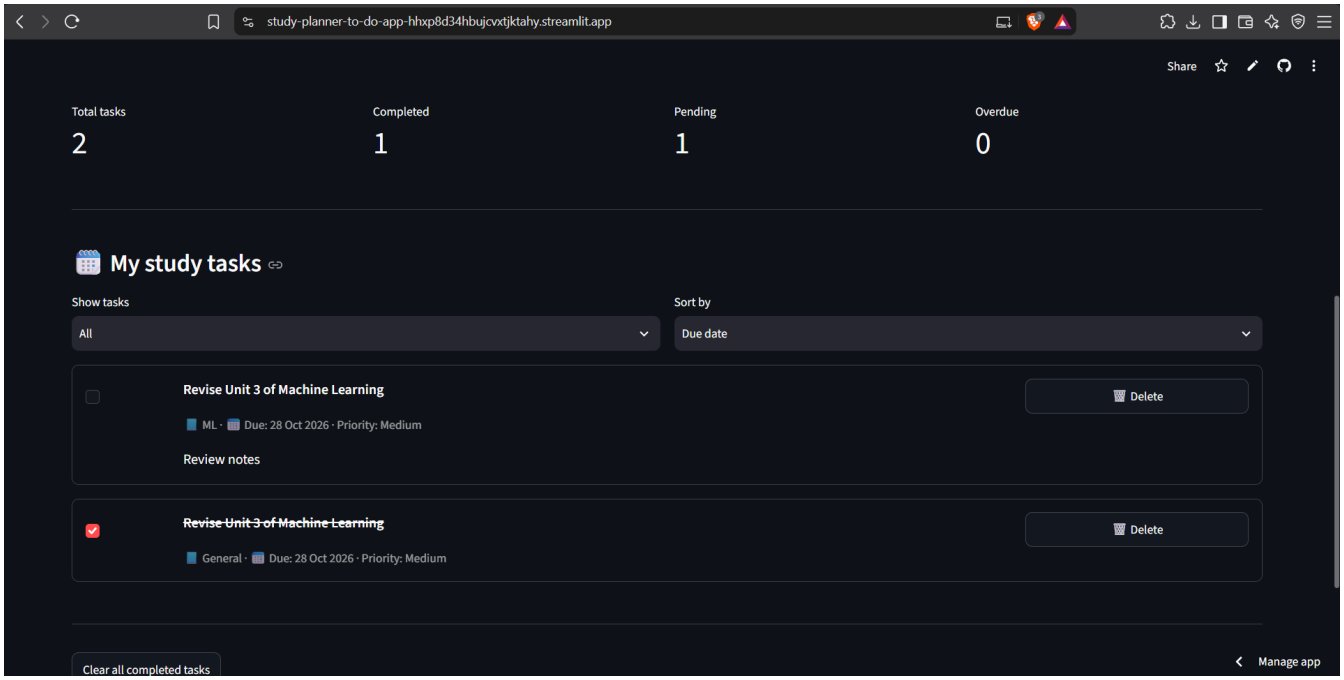


Figure 4. Task list after marking a task as completed.

Observed dashboard state in this screenshot: 2 total tasks, 1 completed, 1 pending, and 0 overdue.

Screenshot 5

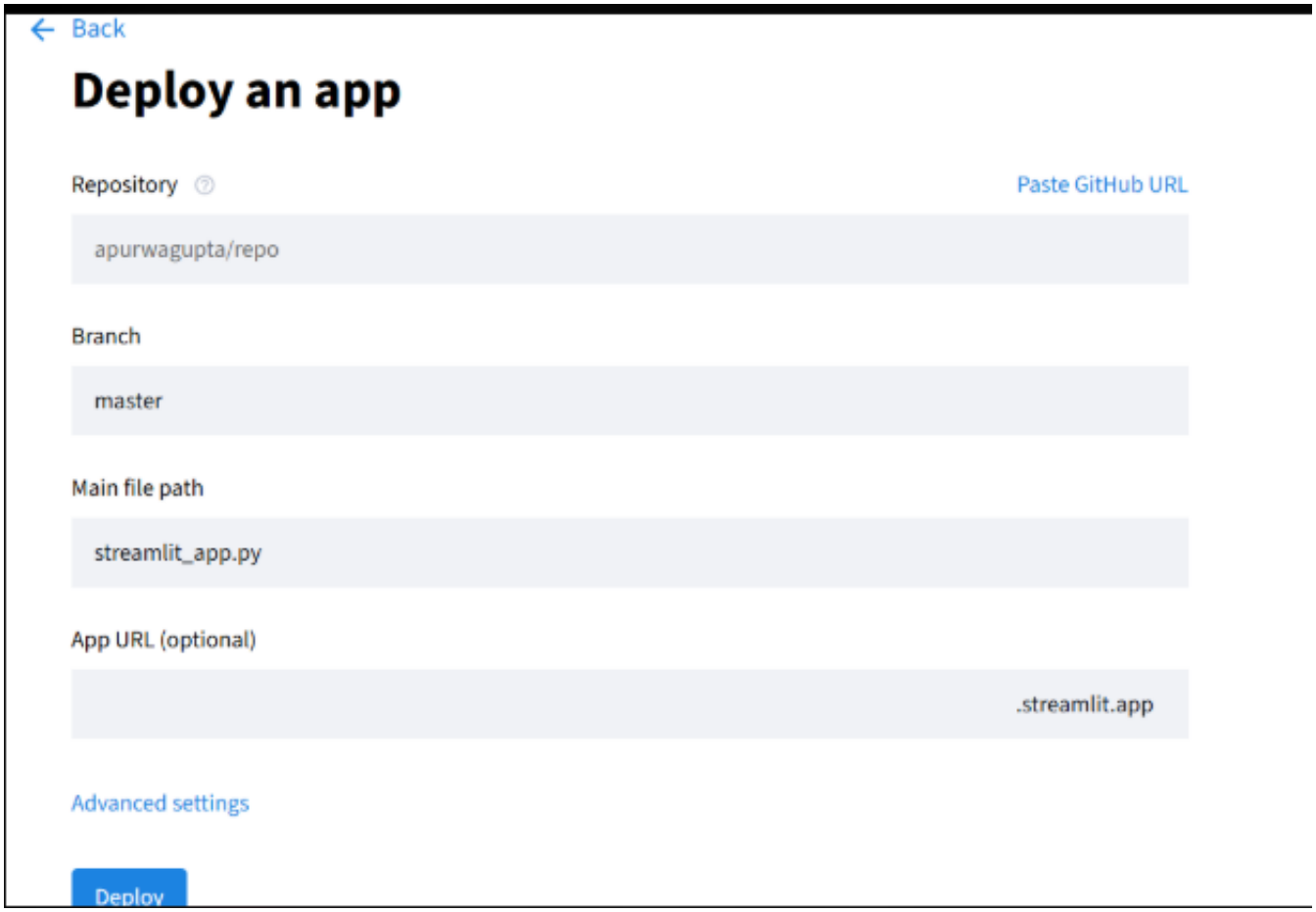


Figure 5. Streamlit deployment configuration page.

7. Cloud Computing Concepts

IaaS (Infrastructure as a Service): Provides fundamental computing resources such as virtual machines, storage, and networking. The customer manages the operating system and application stack.
PaaS (Platform as a Service): Provides a managed platform for developing and running applications, reducing the need to manage servers and operating systems.
SaaS (Software as a Service): Provides a complete software product over the internet, usually accessed through a browser or app.

This project is closest to a PaaS-style deployment experience because the developer supplies application code and dependencies while the hosting platform manages much of the runtime and infrastructure. The app itself is software delivered through a browser, but the deployment activity demonstrates the managed platform model.

8. Example Cloud Free-Tier Comparison

Cloud providers revise free offers, quotas, and eligibility conditions periodically. The table below is a high-level comparison; check each provider's official pricing and free-tier pages before using a service or entering billing details.

Provider	Examples of free offers	Things to check
Amazon Web Services (AWS)	Selected services may offer free usage allowances or short-term trials.	Service-specific limits, account eligibility, duration, and possible charges after limits.
Microsoft Azure	Eligible new customers may receive credits and access to selected free services.	Credit expiry, eligible services, regional availability, and pay-as-you-go conversion.
Google Cloud Platform (GCP)	Eligible new customers may receive trial credits and selected no-cost usage.	Trial period, credit use, product quotas, and billing-account requirements.

9. Testing

The supplied screenshots show the app loading in a browser, accepting task details, displaying the task list and summary counts, and reflecting a completed task. These are manual functional checks. They do not constitute exhaustive testing of all browser, network, or deployment failure conditions.

10. Limitations and Future Scope

The current version stores tasks in Streamlit session state. Task data may be lost when a session resets or the app restarts, and the app does not provide a persistent multi-user database. Future improvements could add SQLite or a managed cloud database, user authentication, recurring reminders, calendar views, and export to CSV.

11. Conclusion

The Study Planner & To-Do App demonstrates how a Python web application can be built with Streamlit and published using a managed cloud hosting platform. It provides basic study task management and progress tracking while illustrating the relationship between source control, application dependencies, cloud deployment, and browser-based access. Adding persistent storage would be the next major step toward a production-ready version.

References

• Streamlit documentation: https://docs.streamlit.io/
• Streamlit Community Cloud: https://streamlit.io/cloud
• AWS Free Tier: https://aws.amazon.com/free/
• Microsoft Azure free services: https://azure.microsoft.com/free/
• Google Cloud Free Program: https://cloud.google.com/free